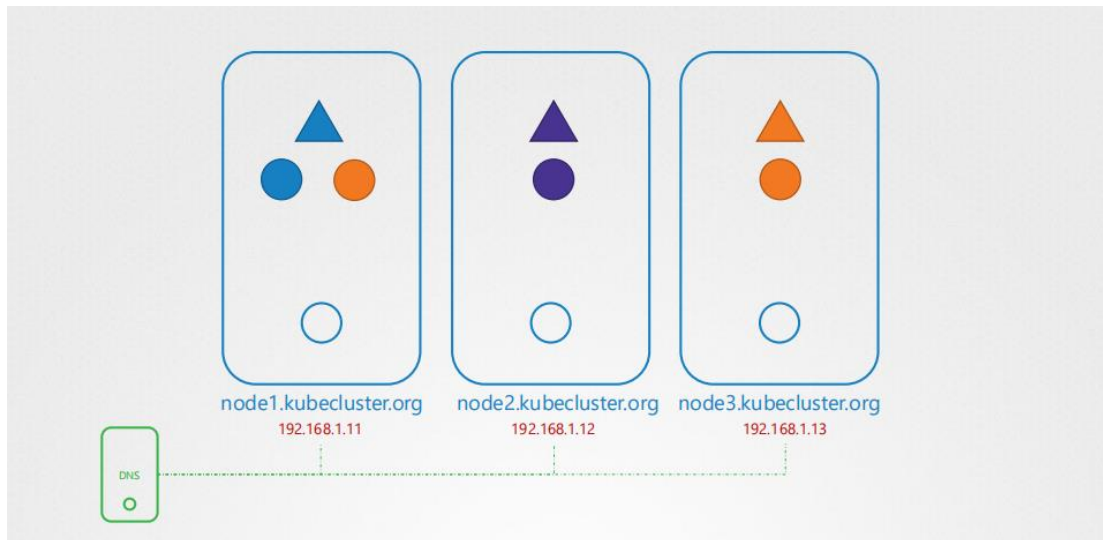


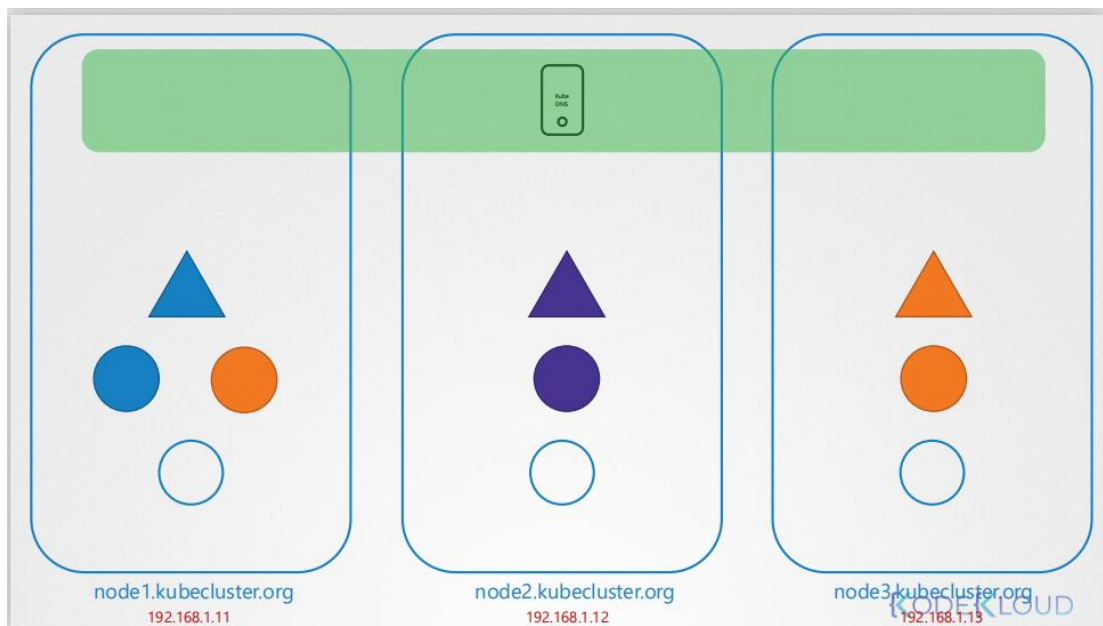
When a service is creating a DNS, entry is added automatically

DNS (name resolution) in K8s cluster

UC: We have a 3 node k8s cluster with some pods, services deployed on them, each node has a node name and ip address assign to it, the node name and ip addresses of the nodes are registered in DNS server.

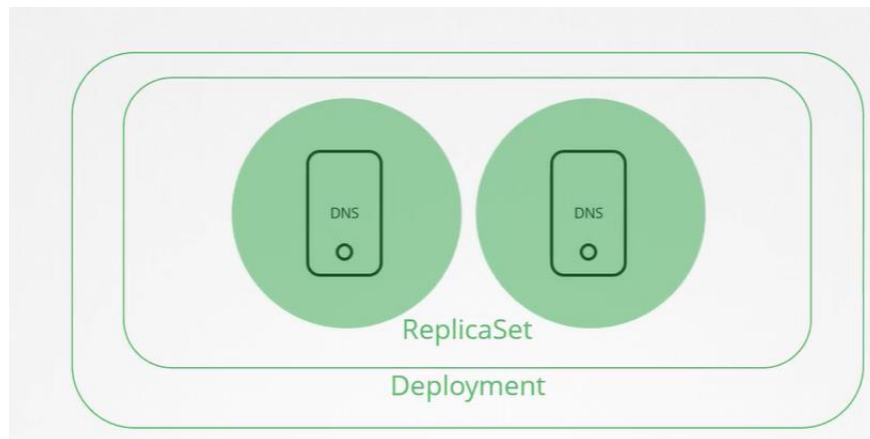


K8s deploys a build in DNS server by default when you setup a cluster.



How kubernetes implements the DNS.

coredns server is deployed as a pod in kube-system namespace



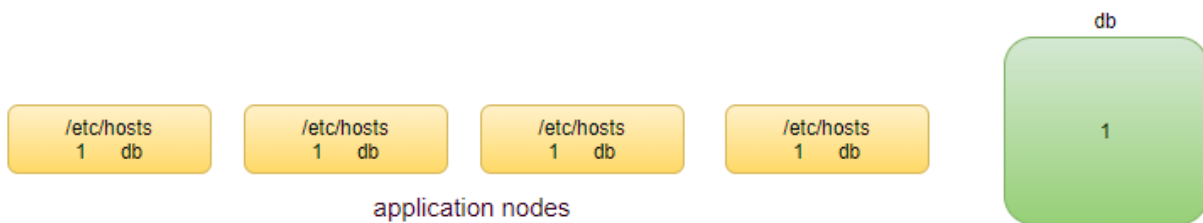
```
root@controlplane:~# kubectl get pods -n kube-system -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
coredns-74ff55c5b-4rp15             1/1     Running   0           87m   10.32.0.3       controlplane        <none>            <none>
coredns-74ff55c5b-ksrck             1/1     Running   0           87m   10.32.0.2       controlplane        <none>            <none>
etcd-controlplane                   1/1     Running   0           87m   10.150.0.19     controlplane        <none>            <none>
kube-apiserver-controlplane         1/1     Running   0           87m   10.150.0.19     controlplane        <none>            <none>
kube-controller-manager-controlplane 1/1     Running   0           87m   10.150.0.19     controlplane        <none>            <none>
kube-proxy-cfr66                    1/1     Running   0           87m   10.150.0.19     controlplane        <none>            <none>
kube-proxy-gtgbm                    1/1     Running   0           73m   10.150.0.18     node02              <none>            <none>
kube-proxy-hcjwj                    1/1     Running   0           68m   10.150.0.20     node01              <none>            <none>
kube-scheduler-controlplane         1/1     Running   0           87m   10.150.0.19     controlplane        <none>            <none>
weave-net-8jcmx                     2/2     Running   1           68m   10.150.0.20     node01              <none>            <none>
weave-net-fzvtm                     2/2     Running   1           85m   10.150.0.19     controlplane        <none>            <none>
weave-net-zggjl                     2/2     Running   1           73m   10.150.0.18     node02              <none>            <none>

root@controlplane:~# kubectl get svc -n kube-system -o wide
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE   SELECTOR
kube-dns  ClusterIP  10.96.0.10    <none>        53/UDP,53/TCP,9153/TCP 124m  k8s-app=kube-dns

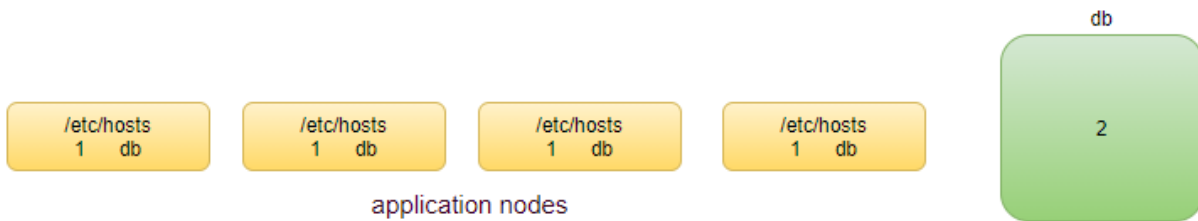
root@controlplane:~# kubectl get deploy -n kube-system -o wide
NAME      READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES               SELECTOR
coredns   2/2     2            2           126m  coredns      k8s.gcr.io/coredns:1.7.0  k8s-app=kube-dns

root@controlplane:~#
```

/etc/host



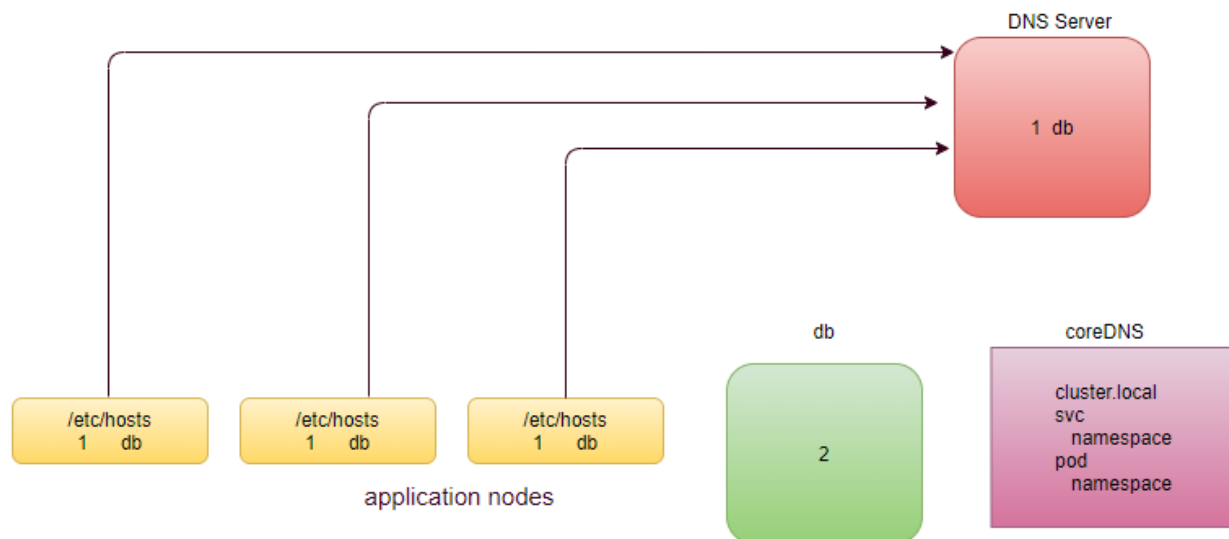
UC: If IP of dbserver changes?



updating the `/etc/host` file is very time-consuming.

We create a DNS server

`/etc/resolv.conf`

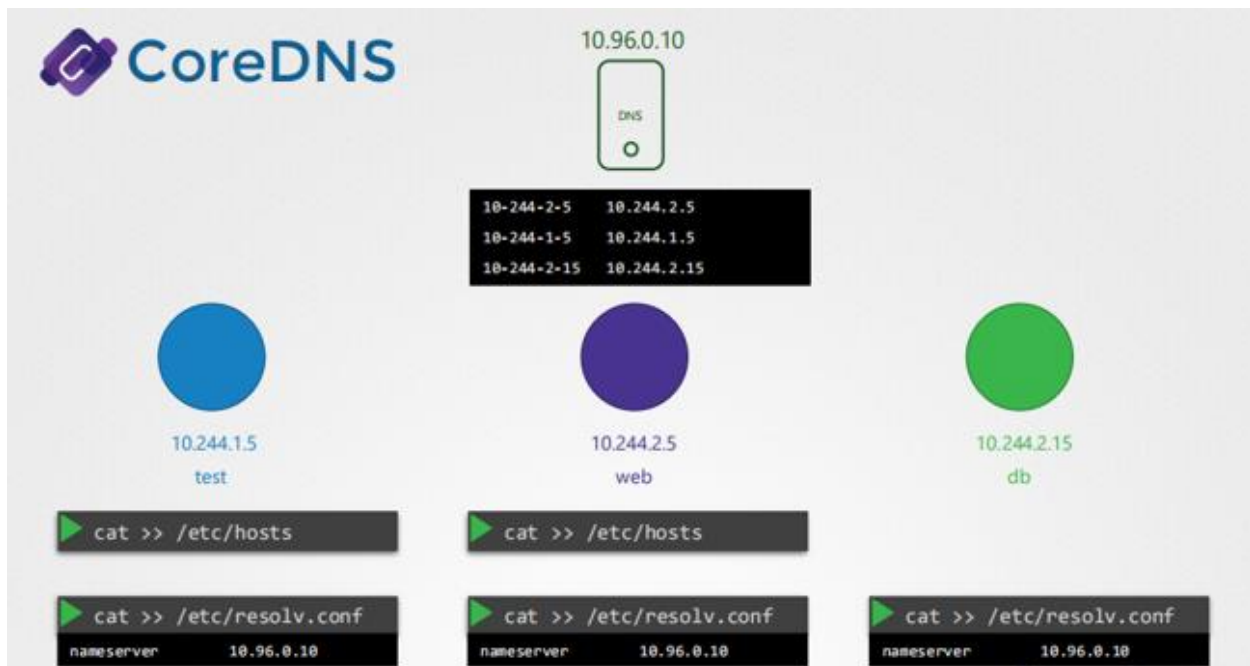


```
root@controlplane:~# more /etc/resolv.conf
# This file is managed by man:systemd-resolved(8). Do not edit.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "systemd-resolve --status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs must not access this file directly, but only through the
# symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a different way,
# replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0
search us-east4-c.c.striking-decker-302009.internal c.striking-decker-302009.internal google.internal
```

When pod is created, same time an ip address generated by weave network assigned to the pod and at same time api-server is going to update etcd db, going

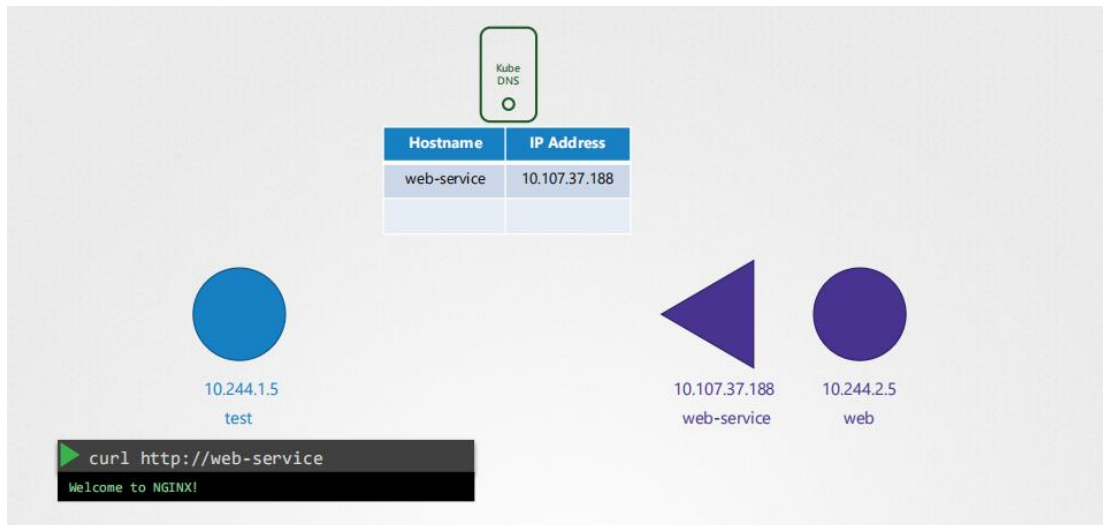
to register with default core dns server. Pod can be reached out using its ip address or its dns name.



Within every pod there is a DNS server entry

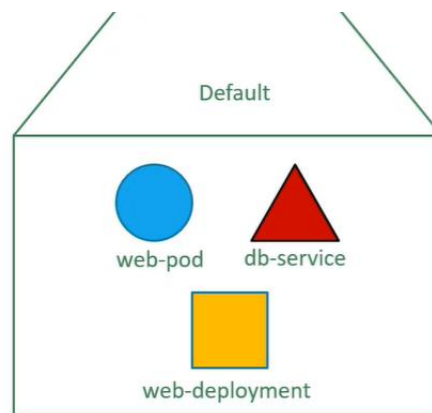
```
root@controlplane:~# kubectl exec webdep-7f6d6dcb79-7xnp6 -- cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local us-east4-c.c.striking-decker-302009.internal c.striking-decker-302009.internal google.internal
options ndots:5
root@controlplane:~# kubectl get svc -n kube-system -o wide
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE    SELECTOR
kube-dns  ClusterIP 10.96.0.10    <none>         53/UDP,53/TCP,9153/TCP 143m   k8s-app=kube-dns
```

UC: We have two pods, test and web pod to access web pod we create a service. Whenever service is created k8s **DNS service creates a record for the service, it maps the service name to the ip address.**



In this case pods and services are all in the same namespaces, you are able to reach webservice simply by its name.

Just similar to family members can reach to one-another by their name, resources in one namespace can reach to one-another by their name, e.g., if web application in default namespace wants to access db-service in **default namespace** can be accessed through their service name directly.

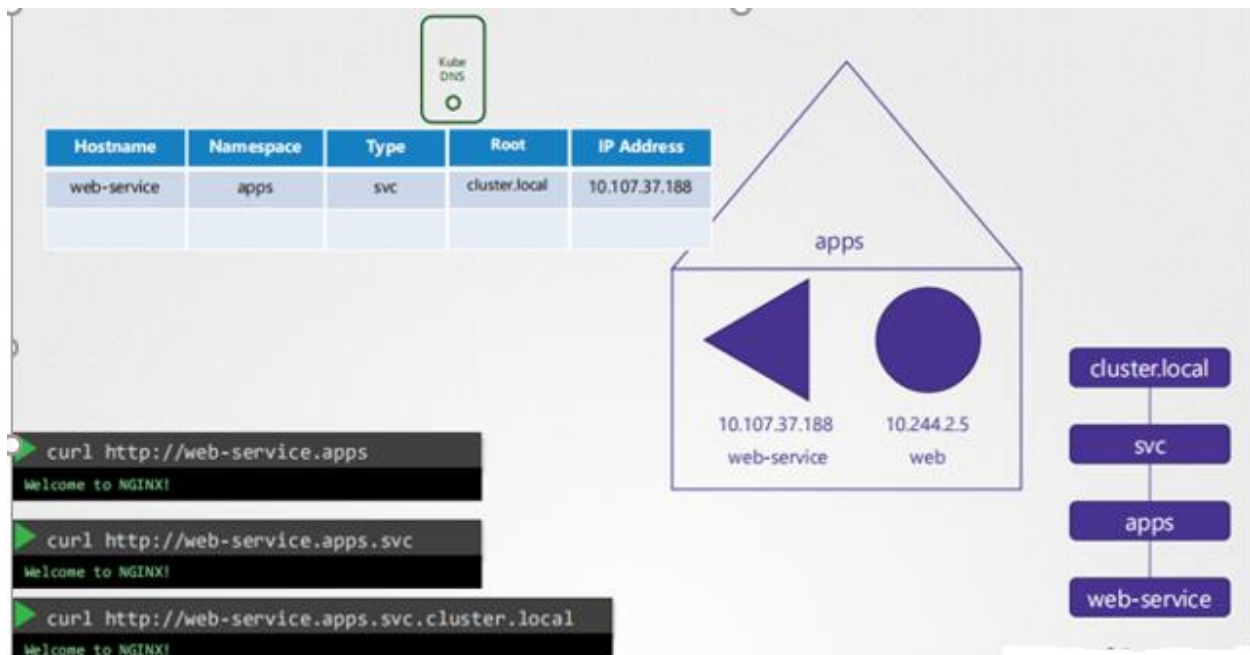


```
mysql.connect("db-service")
```

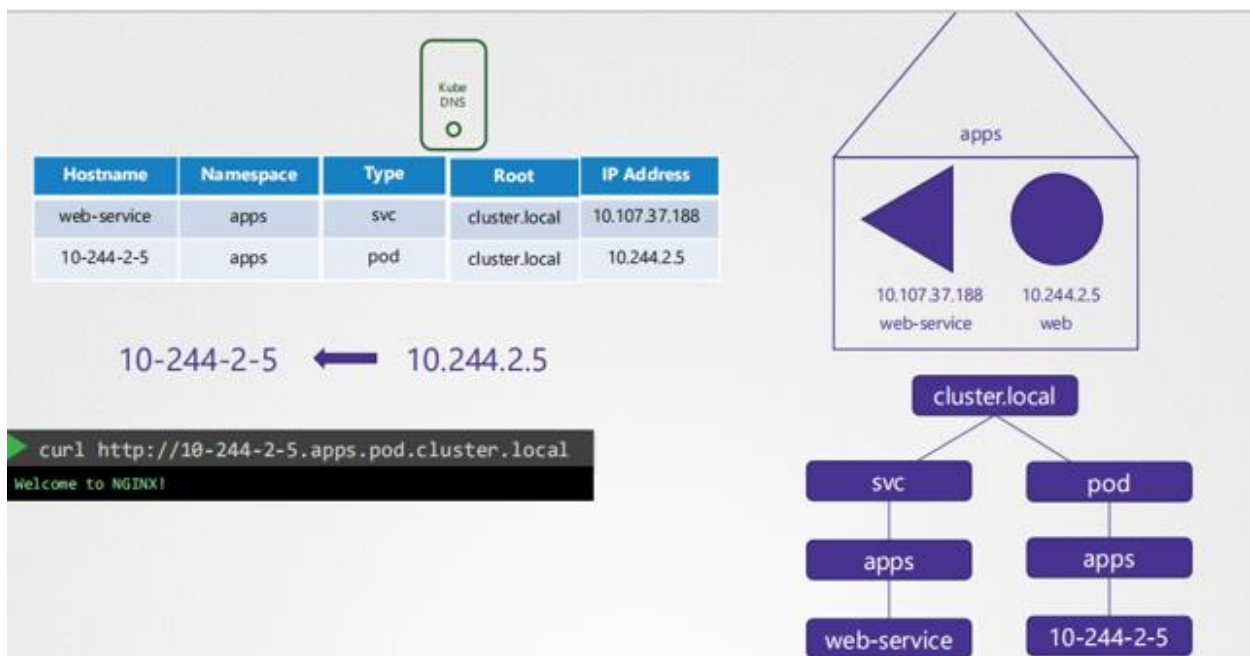
UC: web pod and service are in different namespace? The last name of the service will be its namespace.

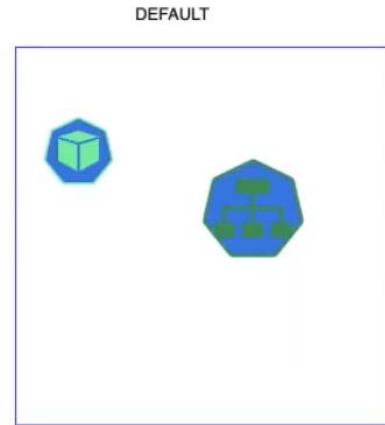
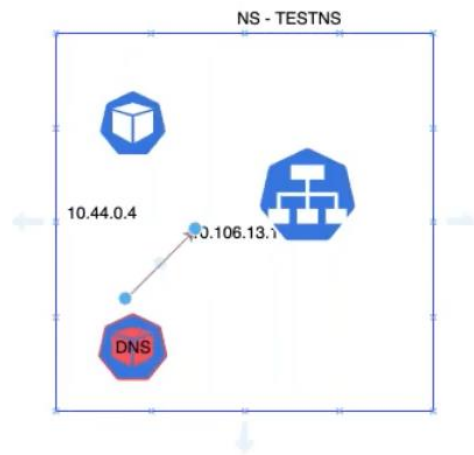
```
mysql.connect("db-service.dev.svc.cluster.local")
```





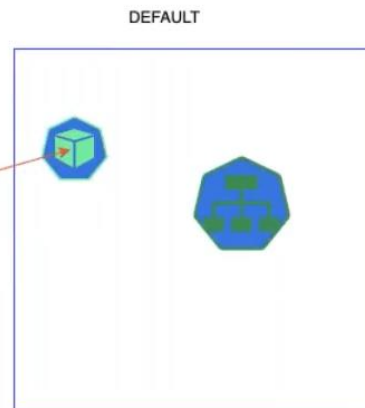
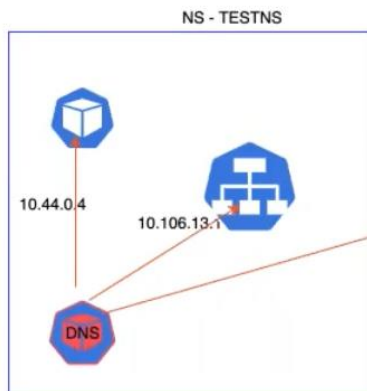
For pods





```
root@masternode:~# kubectl create namespace testns
namespace/testns created
root@masternode:~# kubectl -n testns run nginx --image=nginx --expose --port=80
service/nginx created
pod/nginx created
root@masternode:~# kubectl get all -n testns -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP          NODE   NOMINATED NODE   READINESS GATES
pod/nginx     1/1     Running   0           79s   10.44.0.4   node2   <none>            <none>
service/nginx  ClusterIP   10.106.13.161   <none>    80/TCP    79s   run=nginx
```

```
root@masternode:~# kubectl -n default run mynginx --image=nginx --expose --port=80
service/mynginx created
pod/mynginx created
root@masternode:~# kubectl -n testns run dnstest --image=lerndevops/netshoot --rm -it -- /bin/bash
```



```
bash-5.0# cat /etc/resolv.conf
nameserver 10.96.0.10
search testns.svc.cluster.local svc.cluster.local cluster.local us-central1-a.c.angular-amp-276615.internal c.angular-amp-276615.
internal google.internal
options ndots:5
bash-5.0#
```

```
bash-5.0# nslookup nginx.testns.svc
Server:      10.96.0.10
Address:     10.96.0.10#53

Name:   nginx.testns.svc.cluster.local
Address: 10.106.13.161

bash-5.0#
```

```
bash-5.0# nslookup mynginx.default.svc
Server:      10.96.0.10
Address:     10.96.0.10#53

Name:   mynginx.default.svc.cluster.local
Address: 10.97.103.203

bash-5.0#
```

```
bash-5.0# nslookup 10-44-0-4.testns.pod
Server:      10.96.0.10
Address:     10.96.0.10#53

Name:   10-44-0-4.testns.pod.cluster.local
Address: 10.44.0.4

bash-5.0#
```

kubectl get ep kube-dns -n kube-system

kubectl exec <pod name> -- nslookup <service-name>.default.svc.cluster.local

core dns just register the Pod or service, not the node.